ShopOS - Parts Inventory Management System

Professional Project Documentation

Project Name: ShopOS **Role:** Full Stack Developer / Software Engineer **Application Type:** Enterprise Resource Planning (ERP) / Inventory Management System **Domain:** Auto Parts Retail & Logistics

---

1. Executive Summary

ShopOS is a comprehensive, full-stack inventory management system tailored specifically for auto parts shops. It bridges the gap between traditional retail operations and modern web technologies by leveraging a React/Vite frontend, a robust Node.js/Express backend, and a highly accessible Google Sheets based database architecture. The system is designed for high reliability, real-time performance, and ease of use, enabling administrators and employees to manage large inventories, track sales, generate reports, and monitor system activity seamlessly.

2. Software Architecture & Tech Stack

The application employs a decoupled client-server architecture, enabling independent scaling and seamless deployment pipelines.

Frontend Engineering (Client-Side)

- **Framework:** React 19 + TypeScript + Vite
- **Styling:** Tailwind CSS (Utility-First Responsive UI)
- **Data Visualization:** Recharts for dynamic reporting and analytics
- **Routing:** React Router v7 for client-side Single Page Application (SPA) routing
- **State Management & Context:** React Context API (Auth, Offline capability)
- **Key Modules:**
  - **Dashboard (Home.tsx):** High-level overview of critical metrics.
  - **Inventory (Inventory.tsx, AddItem.tsx, LowStock.tsx):** Full CRUD for parts management with low-stock alerts.
  - **Sales & Processing (BatchSales.tsx, SalesHistory.tsx):** Bulk point-of-sale processing and historical transaction tracking.
  - **Reporting (Reports.tsx):** Visual breakdown of revenue and inventory valuation.
  - **Administration (Settings.tsx, AuditLog.tsx, ActiveSessions.tsx):** System configuration, user session management, and comprehensive audit trails.

Backend Engineering (Server-Side)

- **Framework:** Node.js + Express.js
- **API Architecture:** RESTful API design
- **Database Layer:** Google Sheets API (NoSQL-like implementation)
  - *Data Tables:* `INVENTORY`, `USERS`, `SESSIONS`, `SALES`, `SETTINGS`, `AUDIT_LOG`
- **Authentication:** Custom Session-based Auth with cryptographic signature and expiration.
- **Middleware:** Role-based access control (RBAC), API validation, Error Handling, and CORS configuration.
- **Key Modules:**
  - `auth.js` - User authentication, session creation, and role verification.
  - `inventory.js` - Complex inventory mutations, search, and stock tracking.
  - `sales.js` - Transactional processing and inventory reconciliation.

- `audit.js` / `reports.js` - Data aggregation and system accountability.

Infrastructure & Deployment

- **Hosting / CI/CD:** Vercel/Netlify for Static Frontend, Render/Railway/VPS for Node.js API Service.
- **Cloud Integrations:** Google Cloud Platform (GCP) IAM & Service Accounts for secure API authorization.
- **Process Management:** PM2 for backend process monitoring and automatic restarts.

---

3. Key Technical Challenges & Solutions

A. Stateless vs. Stateful Database Representation

- **Challenge:** Using Google Sheets as a database required handling asynchronous API timeouts and strict rate limits, while ensuring data consistency during concurrent writes (e.g., batch sales).
- **Solution:** Implemented a robust service layer (`services/sheets.js`) with structured caching, batch processing techniques, and retry mechanisms. All data models were properly mapped using UUIDs for reliable row-level identification.

B. Security & Auditability

- **Challenge:** Dealing with multiple employee roles and ensuring accountability for voided sales or modified inventory records.
- **Solution:** Engineered a complete RBAC system integrated with an immutable `AuditLog` . Every destructive action (create, update, delete) emits an event that is recorded with the user's UUID, timestamp, and action context.

C. Complex State & Real-Time Feedback

- **Challenge:** Providing instant feedback for POS (Point-of-Sale) operators performing branch sales without waiting for slow database round-trips.
- **Solution:** Used Optimistic UI updates in React. The frontend assumes a successful transaction, updating local state instantly, while silently reconciling with the Express API in the background.

---

4. Features & Capabilities

- **Intelligent Inventory Tracking:** Real-time stock decrements, low-stock threshold calculations, and quick-add functionality for fast-paced retail environments.
- **Advanced Analytics Engine:** Dynamic financial reports mapping daily/monthly sales trends, profit margins, and inventory valuation utilizing `Recharts` .
- **Session Management & Security:** Secure login flows with active session invalidation (ability for admins to remotely log out users).
- **Extensible Settings Manager:** Database-driven dynamic UI settings, eliminating the need for hardcoded environmental configurations for the client.
- **Responsive & Accessible UI:** Tailored specifically for desktop POS terminals and mobile resolutions for warehouse workers.

---

5. Outcomes & Impact

Developed a fully functional, enterprise-ready MVP that eliminates the need for expensive proprietary software (e.g., Oracle, SAP) for small-to-medium auto parts businesses. The project showcases the ability to design an end-to-end full-stack application, handle complex state, integrate with third-party cloud APIs (GCP), and deploy a production-ready system utilizing modern DevOps practices.